Object Oriented Programming

Chapter 2:

Static Members and Namespaces

Objectives

At the end of this Lecture, students will become familiar with:

- **Static fields and how to use it;**
- **The keyword "this";**
- **Static methods and how to call it;**
- **Static constructor and its use;**
- **The static class "Math" and its methods;**
- **Namespaces and their use.**

Outline

- Introduction
- Static fields
- The keyword "this"
- Static Methods
- Static constructor
- The static class "Math"
- Namespaces

Introduction

- In C#, fields, properties, methods, and constructors can be declared as **static**, and classes can also be static;
- To make a member static, we use the keyword "**static**";
- A static member belongs to the class in which it is defined and <u>it does not belongs to any object</u>;
- A static member is used directly without the need to create an object from the class it belongs to, and to use it, it must be prefixed with the name of the class followed by the **dot** character (.).

Static Fields

- Static fields are often used to define constants;
- The static field can be used as a class-specific variable:
  - In this case, any object can read the same value of the static field (or static property), i.e. there is only one copy of the static field (or static property) in the memory during execution;
  - When changing the static field (or static property), the change will be noticed by all objects of the class;
  - That is, the value of the static field (or static property) will be shared with all objects of the class.

```
public static int instances = 0; // static field, it is public for testing purpose only,
```

The keyword "this"

- The keyword "this" is used within the class as a reference to the current instance of the class to use its members;
  - It is used to avoid the ambiguity of using input arguments having the same names of fields (or properties)

```
1. public class Employee {
2.             private int salary;
3.             private string name;
4.              public Employee(string name, int salary)
5.             {
6.                 // Use this to qualify the members of the class
7.                 // instead of the constructor parameters.
8.                 this.name = name;
9.               this.salary = salary;
10.             }
11.         }
```

In the source code, **name** is the name of a field, and it is also the name of an input argument for the constructor who will assign the argument value to the field. **salary** is the name of a field and the name of an input argument.
In lines 8 and 9, there is no ambiguity because of using “this”.

Static Methods

- A method is declared static using the **static** keyword;
- A static method should use only static members;
- A static method cannot use "this" keyword because it is used by the object as a reference to itself.

```
private string name;
public static string Organisation { get; set; }  // static property
public int GetInstances( ) {
    return instances;
    }// end GetInstances

 public static void WritePersonNumber( ) {
    Console.WriteLine("Using WritePersonNumber, Person Counter is : {0} ", instances);
     //   name = "abcd"; //Not accepted
     //   this.name = "abcd"; //Not accepted
     }// end WritePersonNumber
    // constructor
 public Person(string name) {
     instances++;
     this.name = name;
      }//end constructor
```

Tutorial Example

```
using System;
namespace StaticMembers {
class Person {
    public static int instances = 0; // static field, it is public for testing purpose only,
    private string name;
    public static string Organisation { get; set; }  // static property
    public int GetInstances( ) {
        return instances;
        }// end GetInstances

    public static void WritePersonNumber( ) {
      Console.WriteLine("Using WritePersonNumber, Person Counter is : {0} ", instances);
        //   name = "abcd"; //Not accepted
        //   this.name = "abcd"; //Not accepted
        }// end WritePersonNumber
```

```
 // constructor
public Person(string name) {
        instances++;
        this.name = name;
         }//end constructor

    public string Name  {
        get { return name; }
        set { name = value; }
         }// end Name
  }// end class Person
```

Tutorial Example

```
1.  class PersonTest {
2.      static void Main( )  {
3.          Person person1 = new Person("Ahmad");
4.          Person person2 = new Person("Ziad");
5.          Person person3 = new Person("Reem");
6.          Person.Organisation = "Univ.";
7.          // using the class name (Person) to call the value of instances
8.          Console.WriteLine("Using instances, Person Counter is : {0} ",
    Person.instances);
9.          // if you use the first object (Person1) to call the value of
10.         // instances like this:
11.         //  Console.WriteLine("Person Counter is : {0} ", person1.instances);
12.         // you will get the error message:
13.         // "Member 'Person1.instances' cannot be accessed with an instance
14.         //  reference qualify it with a type name instead
15. Person.WritePersonNumber();
16. //   person1.WritePersonNumber();// Not accepted
17. Console.WriteLine("Person Organisation is : {0} ", Person.Organisation);
18. Console.WriteLine("Person1 Counter is : {0} ", person1.GetInstances());
19. Console.WriteLine("Person2 Counter is : {0}
    ", person2.GetInstances());
20. Console.WriteLine("Person3 Counter is : {0}
    ", person3.GetInstances());
21. //Console.WriteLine("Person Counter is : {0} ",
    //Person.GetInstances());
22. // Not accepted: you will get the error message: “an
    //object reference is required for the non-static field,
    //method or property 'Person.GetInstances()”
23. Console.ReadKey();
24.     }// end Main
25.   }// end class PersonTest
26. }  // end  namespace StaticMembers
```

Tutorial Example

- The number of objects is printed once by calling the value of the static field **instances** prefixed with the name of the class,
- It is printed another time by calling the static method **WritePersonNumber** prefixed with the name of the class,
- Then the value of the static property **Organisation** is printed; we have called the property using the name of the class as well;
- The number of objects is printed again by calling the non-static method **GetInstances** with each of the three objects.
- After execution, we get the following output:

```
Using instances, Person Counter is : 3
Using WritePersonNumber, Person Counter is : 3
Person Organisation is Univ.
Person1 Counter is 3
Person2 Counter is 3
Perso31 Counter is 3
```

Tutorial Example

- We note that the value of the **instances** (number of created objects) is (3), and all three objects created have the same value, which is the same value obtained when calling directly the static members. In this case, the static field is treated as a global variable for all objects.
- We also note that if we use an object (e.g. person1) directly to invoke the value of the static field, we will get the following error message:

```
Member 'Person.instances' cannot be accessed with an instance reference; qualify it with a type name instead
```

Which means that the class name (i.e., the name of the data type) should be used to call the static field.

- An object can be used to call the non-static method **GetInstances** that returns the value of the static field **instances**; So, a non-static method can use a static member.

Tutorial Example

- The method **WritePersonNumber** is static and prints on the screen the value of the static field **instances**, and when we try to use the field **name** within it, we get the error message:

```
An object reference is required for the non-static field, method, or property 'Person.name'
```

which means that an object must be created in order to use a non-static element.

- But when we try to use the keyword "**this**", we get the error message:

```
Keyword 'this' is not valid in a static property, static method, or static field initializer
```

Which means that this word is not allowed to be used with static properties or with static methods, not even when initializing static fields.

Static Constructor

- A class can have a static constructor, and the static constructor is declared using the keyword **static** as well.
- The static constructor is used to initialize static fields or to do a task that only needs to be done once.
- It is called automatically before the first object of the containing class is created or before the first use of a static class member, and it is sometimes used when writing to Log Files.
- The static constructor has an access specifier, it has no input argument, it cannot be overloaded, it should not be inherited, it should not be called directly, and the class cannot have more than one static constructor.

Static Constructor

```
1.  using System;
2.  namespace StaticConstructor {
3.        class Car {
4.           // * A static constructor is executed only once,
5.           // when a class is first accessed.
6.           // * A static constructor cannot have any access modifiers
7.           // * A static constructor cannot have any parameters
8.           static Car() {
9.               Console.WriteLine("Car initialized");
10.          }// end static constructor
11.
12.          //Instance constructor, this is executed every
13.          //time the class is created
14.          public Car()
15.          {
16.              Console.WriteLine("Car created");
17.          }// end Instance constructor
18.          static void Main( ) {
19.              Car mazda = new Car();
20.              Car toyota = new Car();
21.              Console.ReadKey();
22.          }// end Main
23.       }// end  class Car
}// end namespace StaticConstructor
```

- An example showing how to use the static constructor
- The program output:

```
Car initialized
Car created
Car created
```

Static Class

- A class can be static, and a class is declared static using the keyword **static**, as shown:

[<access modifier>] **static** class <class_name> {
// … Class body goes here
}

- A static class is a class whose elements are all static, and therefore no object can be created from it
- A static class is used to group a set of common-use constants and methods into a single structure
- As an example of this type of class, the static class **System.Math**

Static Class

- The static class **System.Math** contains a set of methods and constants that are used to write various mathematical expressions.
- Among the mathematical constants defined within the Math class are:

PI = 3.14159… and E = 2.71828….

Expression	Method
Returns the absolute value of x	**Abs(x)**
Returns the smallest integral value that is greater than or equal to x.	**Ceiling(x)**
Returns the cosine of the specified angle x.	**Cos(x)**
Returns **e** raised to the specified power.	**Exp(x)**
Returns the largest integral value less than or equal to x	**Floor(x)**
Returns the natural (base e) logarithm of a specified number x.	**Log(x)**
Returns the larger of x and y	**Max(x, y)**
Returns the smaller of x and y	**Min(x, y)**
Returns a specified number raised to the specified power.	**Pow(x, y)**
Returns the sine of the specified angle x.	**Sin(x)**
Returns the square root of a specified number x.	**Sqrt(x)**

Namespaces

- When working as a team, sometimes two programmers may use the same name to denote two different classes. When these two classes need to be used, a Naming Collision will occur;
- To solve this problem, the classes are organized into namespaces, so that the name of a class is used once within its namespace (it is unique);
- Two classes with the same name can be distinguished based on their respective namespaces because they cannot exist together within the same namespace;
- To access the **ClassName** class within the **MyNamespace** you must use the statement **MyNamespace.ClassName**
- If you type the statement **using MyNamespace;** in the beginning of your program, the keyword "**using**" calls the namespace **MyNamespace**, then the **ClassName** can be used directly without being preceded by the name of the namespace;
- A namespace is an assemblage of the names of several classes together without taking into account how the classes are stored in the file system. So, a namespace can exist within a single file, or its classes can be distributed over several files.

Namespaces

```
1. using System;
2. namespace NS1   {
3.     namespace Ns2  {
4.         namespace CSharp  {
5.             public class Program  {
6.                 public static void Main( )  {
7.                     Console.WriteLine(typeof(Program).Namespace);
8.                     Console.ReadKey();
9.                 }//end Main
10.             }//end Program
11.         }//end namespace CSharp
12.     }// end namespace N2
13. }// end namespace N1
```

- A file can contain more than one namespace
- Usually, classes in the same namespace have common characteristics and are used in the same context. A namespace can contain other namespaces, so we get what are called **Nested Namespaces.**
- In the program, the NS1 namespace contains the NS2 namespace, which in turn contains the CSharp namespace. The last one contains the class **Program**. In the Main method, the statement **typeof(Program).Namespace** is used to get the name of the namespace containing the class **Program**. After executing the program, we get the following output:

```
NS1.Ns2.CSharp
```

Namespaces

- The two programs are equivalent, the namespaces are separated using the character (.)

```
using System;
namespace NS1  {
    namespace Ns2  {
        namespace CSharp  {
            public class Program  {
                public static void Main( )  {
                    Console.WriteLine(typeof(Program).Namespace);
                    Console.ReadKey();
                }//end Main
            }//end Program
        }//end namespace CSharp
    }// end namespace N2
}// end namespace N1
```

```
using System;
namespace NS1.Ns2.CSharp {
        public class Program  {
        public static void Main( )  {
            Console.WriteLine(typeof(Program).Namespace);
            Console.ReadKey();
        }//end Main
    }//end Program
}// end namespace NS1.Ns2.CSharp
```

Namespaces

- In the FCL Framework Class Library of .NET, predefined classes are organized into several namespaces, some of which are explained in the following table:

Description	Namespaces
It contains the basic classes of most programs. It includes basic data types such as **String**, **DateTime**, etc., in addition to special basic classes such as **Math**, **Exception**, and **Array**. It also contains the class **Console**.	**System**
It contains classes and interfaces that are used to define **lists**, **queues**, **stacks**, **hash tables**, and **dictionaries**	**System.Collections**
It includes classes to deal with files and data streams, and contains data types that are useful for dealing with file systems. Among them: **File**, **Directory**, **StreamReader**, **StreamWriter**, **FileStream**, **BinaryReader**, **BinaryReader**	**System.IO**
It includes classes that support network protocols, such as: **HTTP**, **FTP**, and **SMTP**	**System.Net**

Namespaces

Description	Namespaces
It includes the classes and interfaces responsible for providing data security and controlling access to various software components	**System.Security**
It contains classes for handling ASCII and Unicode, classes for converting between character blocks and byte blocks, and classes for processing and formatting strings	**System.Text**
It includes classes and interfaces used in multi-threaded programming	**System.Threading**
It includes classes and interfaces for working with XML	**System.Xml**

Summary

- Static fields and static methods
- The keyword "this"
- Static Constructor
- The static class "Math"
- Namespaces

Thank you very much for attending

You can kindly ask questions